

**Comparative Analysis of Sequential Models for One-Step-Ahead Mobile Network Traffic
Forecasting**

Shima Serein

Machine Learning Techniques I

September 2026 Term

20 September 2026

Abstract

Capacity assignment, overnight base-station sleep scheduling and anomaly alarms all act on a ten-minute horizon, and each acts on a forecast of the next slot. This study compares three architecturally distinct sequential models for one-step-ahead forecasting of mobile Internet traffic on the Telecom Italia Milan grid, and asks whether their ranking survives a change of geographical area. A streaming aggregation reduces 19.4 GiB of Call Detail Record text to a 357 MB memory-mapped matrix at a peak of 109 MB above baseline per daily file, a fifth of a naive load. A long short-term memory network, a dilated temporal convolutional network and gradient-boosted regression trees are trained per area on an identical 144-slot window and evaluated on 16 to 22 December against persistence, seasonal naive and linear autoregressive baselines. The ordering of the three architectures reverses between cells 500 m apart, and that reversal is the one result a predictive-accuracy test supports: the convolutional network beats the recurrent one on one cell and loses to it on another, jointly at $p = 0.004$. Its 3.4 and 3.5 percent leads over a 145-parameter linear autoregression are not separable from zero ($p = 0.36$ and 0.32), while on the third cell that linear model beats all three trained architectures on every metric, by 7.1 percent of the trained model's error ($p = 0.0009$). No architecture betters it on average, it trains in 0.02 s against 44 s, and persistence alone reaches R^2 0.99. The calendar features that help seven of nine model-area pairs cost the convolutional model 13 percent on the cell whose working-day contrast is weakest, the reported failure case.

Keywords: time series forecasting, mobile network traffic, sequential models, call detail records, temporal convolutional network, forecast evaluation

1. Introduction

A mobile operator reads its network in ten-minute slots. Capacity assignment to a cell, the decision to put a base station into a sleep mode overnight to save energy, and the alarm that flags an outage or a flash crowd all act on that horizon, and each acts on a forecast of the next slot [1], [2]. A forecast at 10-minute resolution decides how much capacity to switch on before demand arrives, and how much to switch off when it will not.

This study takes that one-step problem to the Milan Call Detail Record aggregates released for the Telecom Italia Big Data Challenge [3]: 62 days of Internet activity for 10 000 grid cells at 10-minute resolution, 319 896 289 raw rows in 19.4 GiB of text. The research question is fixed by the brief and is quoted verbatim:

How do different sequential models compare for one-step-ahead mobile network traffic forecasting, and how does their performance vary across geographical areas with different traffic characteristics?

The objective is not the model with the lowest error. It is to defend the decisions around three architecturally distinct models run under one protocol on three areas, and to explain the differences from the data. Two measurements set the terms. First, persistence, the forecast $\hat{y}(t+1) = y(t)$, already reaches R^2 0.99 on the highest-traffic cell, and the best linear baseline lowers its mean absolute error by only 8.4 percent. Second, the three highest-traffic cells, squares 5161, 5059 and 5259, lie within about 500 m of each other yet run on different weekly regimes, with weekday-to-weekend traffic ratios of 0.79, 1.29 and 2.98. The second half of the research question is therefore a question about neighbours.

2. Related Work

2.1 One-step forecasting is not where this dataset has been studied

The Milan release is a dataset, not a benchmark [3], and the forecasting work built on it has run towards longer horizons and spatial structure. Zhang and Patras forecast the whole grid at horizons up to ten hours, reporting up to 61 percent lower NRMSE than Holt-Winters, ARIMA, MLP and SVM, the margin growing with the horizon [2]; Chen et al. judge a multivariate LSTM on the same two months by a downstream planning objective [4]; and operator data from other cities points the same way [5], [1]. The closest short-horizon neighbour, STDenseNet, is citywide and spatial where this study is per-cell and univariate [6], and that is the scope decision: holding the input to one cell's own history makes the comparison about temporal inductive bias rather than spatial context. Trinh et al. are closest to the present setting, a single-cell LSTM at one and fifteen steps ahead [7]. This study fixes the horizon at one 10-minute slot, models three cells separately, and tests on 16 to 22 December instead of the window of [2], which straddles Christmas.

2.2 Recurrence, convolution, and a model with no sequence bias at all

The LSTM cell holds information across long lags through gated constant-error carousels, demonstrated on minimal lags in excess of 1000 steps [8]; a 144-step window sits well inside that reach, so if the recurrent model degenerates here, memory is not the explanation. The convolutional alternative makes memory an explicit parameter instead of a learned property: Bai, Kolter and Koltun stack dilated causal convolutions in residual blocks, reporting that this generic architecture outperforms recurrent networks across the standard benchmark suite [9]. The third family drops sequence structure entirely: gradient boosting fits additive expansions of regression

trees by steepest descent in function space [10], and Elsayed et al. feed such a model a flattened window of lagged targets to beat eight deep forecasters, the gain coming from the windowing rather than from the trees [11]. That result dictates the comparison: the tree ensemble receives the same 144 lags and calendar features as the two networks, so any gap is attributable to inductive bias.

2.3 The case that simple methods are hard to beat

Makridakis et al. ran ten machine-learning methods against eight statistical ones on 1045 monthly M3 series under one protocol; the six most accurate were statistical, and at one step ahead an MLP scored 8.39 percent sMAPE against 7.19 percent for ARIMA [12]. Zeng et al. make the same argument against Transformers: a single linear layer from look-back window to forecast beats five of them on all nine benchmarks [13]. The measurement here agrees: persistence alone reaches R^2 0.9902 and WAPE 6.42 percent on square 5161. A linear AR over the same 144 lags is therefore reported in every table, so a nonlinear model must beat a linear map of its own inputs and not merely the previous value. Transformers were excluded on this evidence and on their cost over a univariate 10-minute series.

2.4 Measuring accuracy when the series never repeats its scale

Percentage errors degenerate near zero and scale-dependent errors cannot be pooled across series, so Hyndman and Koehler propose the mean absolute scaled error [14]; accuracy is assessed with a rolling forecasting origin [15]. MASE is therefore reported with a training-period denominator rather than one taken from the window being scored, a distinction Section 5.10 quantifies. Seasonal naive is run at both the daily lag of 144 and the weekly lag of 1008, which is how the weekday regime of square 5259 surfaced, at MAE 470.3 against 210.2.

2.5 The gap

Work on the Milan release concentrates on spatio-temporal architectures at multi-hour horizons [2], [4]; the 10-minute one-step horizon belongs to the regime the critical literature describes [12], [13], [11]. No study on this dataset compares a recurrent model, a convolutional model and a tree ensemble on identical inputs, against persistence, seasonal-naive and linear baselines, across cells whose weekday-to-weekend ratios span 0.79 to 2.98, using scale-free errors and recorded compute cost.

3. Dataset and Data Preparation

3.1 Provenance and schema

The data are the Milan Call Detail Record (CDR) aggregates published for the Telecom Italia Big Data Challenge [3], [16]. Milan is covered by a 100×100 grid of cells about 235 m on a side [17], with activity aggregated into 10-minute slots from 1 November 2013 to 1 January 2014. The release is 62 daily tab-separated files, 19.4 GiB and 319 896 289 rows. Each row is one (cell, slot, country) triple: `square_id` (1 to 10 000), `time_interval` (epoch milliseconds, CET), `country_code`, then `sms_in`, `sms_out`, `call_in`, `call_out` and `internet`, the target. An empty field is a missing record, not a zero.

3.2 What the values measure

The values are scaled CDR activity counts published without the scaling constant, so they are comparable across cells and across time but have no physical unit. An Internet CDR is written when a connection starts or ends and, during a session, every 15 minutes or every 5 MB [3], so the series measures connection activity more than transferred volume.

3.3 Reduction to a memory-mapped store

The research question needs traffic per cell per slot, so `country_code` is summed out, collapsing the 319 896 289 rows into an $8\,928 \times 10\,000$ float32 matrix: 357 MB on disk, a 58-fold reduction. Per-day archives keep all five channels and bring the store to 1.68 GB. The matrix is memory-mapped with NumPy [18], so analysis code slices columns without loading the array.

The reduction is a streaming accumulation using the three levers the pandas documentation gives for larger-than-memory data: fewer columns, narrower dtypes and chunking [19], [20]. Each file is read in chunks of 2 000 000 rows; `country_code` is never parsed; integers are pinned to int32 and floats to float32; each chunk is folded into a preallocated (144, 10 000) float64 accumulator with `np.bincount`. Peak memory follows the chunk, not the file.

3.4 Memory evidence

Table 1 compares three implementations of the one-day aggregation, with peak resident set size sampled every 20 ms above a pre-call baseline. The naive full read needs 559 MB; narrow dtypes and three columns cut that to 313 MB; the chunked accumulator needs 109 MB, 5.1 times less, and is the fastest at 1.41 s. Over the full 62-day ingestion peak RSS never exceeded 377.9 MB and the run took 129.4 s, 2.09 s per day.

Table 1

Peak resident memory above baseline and wall time, three implementations of the one-day aggregation (file of 1 November 2013, 307.9 MiB; Linux aarch64, 4 GiB RAM).

Variant	Peak RSS	Wall time	Day total
A. Naive full read, all columns	559 MB	2.03 s	82 479 247
B. Full read, narrow dtypes, 3 columns	313 MB	1.57 s	82 479 264
C. Chunked accumulation	109 MB	1.41 s	82 479 247

The last column carries a precision warning: variant B sums in float32 and its total differs by 17 units from the float64 sums of A and C. Narrow dtypes are a storage decision, not an arithmetic one; the pipeline accumulates in float64 and casts once.

3.5 Trade-offs and limitations

Summing over country is lossy and irreversible without re-ingesting the 19.4 GiB, so the store answers nothing about roaming. A zero in the store means no row carried a non-missing Internet value for that cell and slot, indistinguishable from genuine silence: 152 527 such pairs, 0.17 percent of the matrix, concentrated in 98 low-traffic cells, five of which record nothing after early December; none of the five modelled cells contains a zero. The matrix is also time-major, so reading one cell's 36 KB series touches about 146 MB of file-backed cache; the process's physical footprint is unchanged, but the layout remains a limitation.

3.6 Hardware

The memory strategy was developed and measured in the 4 GiB sandbox. All model training and every timing in Section 6 ran on an Apple M1 Pro with 16 GB of unified memory under macOS, using PyTorch 2.14 [21] with the MPS backend. No quoted number mixes the two machines.

4. Exploratory Analysis

Every statistic here is computed on the training rows alone, 1 November to 8 December, unless explicitly descriptive of the whole record. The distinction matters: searching lags 1 to 400 on square 5259, the last partial autocorrelation above a practical 0.05 falls at lag 310 on the training slice and at lag 159 on the full 8928 slots, so a sequence length read off the full series would have been set partly by the week it is evaluated on.

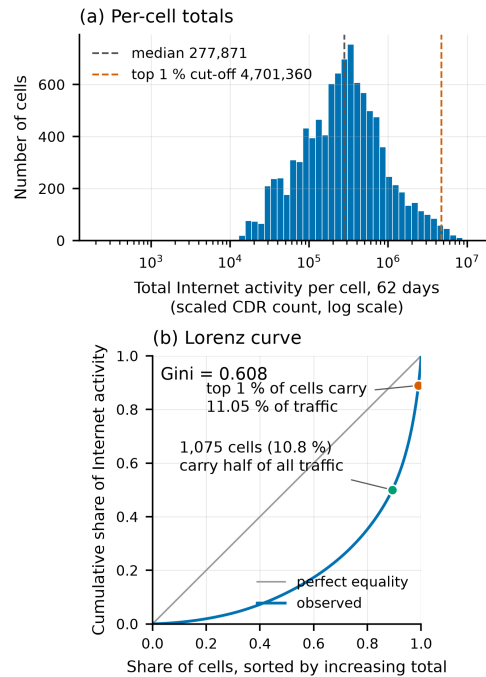
4.1 Spatial concentration

Per-cell Internet totals over the 62 days span almost five orders of magnitude, from 214 to 12 740 060. The top 1 percent of the 10 000 cells carry 11.05 percent of all traffic, the Gini coefficient is 0.608, and 1075 cells reach half the total (Figure 1). On a logarithmic axis the histogram is unimodal and near-symmetric, skewness 0.02, so the concentration is the ordinary consequence of a log-normal spread; the left tail is the longer one, a handful of near-empty cells 3.1 decades below the median against 1.7 above.

The two cells the brief names sit in the upper tail without being extreme: 4556 ranks 109th and 4159 ranks 424th, at 16.5 and 8.8 times the median cell. The three highest-traffic cells are squares 5161, 5059 and 5259, at 12.74, 11.17 and 10.49 million, and the ranking is stable under pre-evaluation totals, totals excluding the Christmas collapse, and the mean over non-zero slots. The fourth cell, 5061, is 8.6 percent below the third, so the boundary is no near-tie.

Figure 1

Total Internet activity per cell over the 62 days. Left: histogram on a logarithmic axis, with the median and the top-1-percent cut-off. Right: Lorenz curve, annotated with the top-1-percent share and the number of cells needed to reach half of all traffic.



The three evaluation areas are the most extreme cells of a heavy-tailed distribution, so nothing measured on them transfers to a typical cell, and their levels differ by orders of

magnitude, making MAE and RMSE comparable only within an area. Section 6 therefore compares areas on the scale-free WAPE and MASE, and every scaler is fitted per area.

4.2 The three areas are neighbours with different regimes

All five study cells lie in the dark central core, inside a window of roughly 2.8 km by 1.4 km (Figure 2). The three top cells sit at grid positions (51, 60), (50, 58) and (52, 58), every pair at Chebyshev distance 2, so no two share an edge or a corner, yet all three lie within about 470 to 525 m of each other. The research question's "geographical areas with different traffic characteristics" therefore means neighbouring cells of one core, the stronger test: whatever separates them is land use at cell scale.

The grid geometry says which land use. The published lattice has square 1 in the south-west corner with ids running east before north [17], so a centroid follows from the id alone, and that orientation is measured rather than assumed: under it the agricultural belt south of the city carries 52 000 activity units against 1 195 000 in Sesto San Giovanni, while the reflected reading would make the farmland the busier of the two, and the traffic mass has its mean row 1.5 km north of the Duomo rather than in open fields. Read that way, square 5161 centres 320 m east-north-east of the Duomo on the Corso Vittorio Emanuele retail axis, square 5059 206 m west-south-west on Via Torino, and square 5259 409 m north-north-west at Cordusio, between La Scala and the Piazza Affari banking district; square 4556 centres on the Darsena at the head of the Navigli, and square 4159 at Porta Lodovica on the edge of the Bocconi campus. The coordinates were resolved against OpenStreetMap, and the land uses are the hypothesis those addresses support rather than a measurement.

What separates them is the weekly regime (Figure 3). Over 1 to 14 November the weekday-to-weekend mean ratio is 0.79 for square 5161, busier at weekends and peaking between 13:30 and 18:30; 1.29 for 5059; 2.98 for 5259, whose weekday mean of 1597 falls to 536 at weekends; 1.93 for 4159; and 0.90 for 4556, which peaks between 19:00 and 23:40 on 13 of the 14 days. Pearson correlation over the fortnight orders the pairs the same way: 0.84 for 5161 and 5059, 0.78 for 5059 and 5259, but 0.43 for 5161 and 5259.

On square 5259 the weekday pattern does not shrink at weekends, it disappears. The daily maximum falls between 13:00 and 17:20 on all nine working days of the fortnight and between 22:50 and 02:10 on all five non-working days, the weekends plus Friday 1 November, the Ognissanti public holiday. The regime follows the working calendar and not the day-of-week label, which is why Section 5's calendar features encode a working-day indicator with the window's holidays marked, in place of a weekend flag. Square 4159 reproduces the same shape at a fifth of the amplitude, a daily-profile correlation of 0.980 against 5259: an amplitude contrast, not a second regime.

Every one of the five regimes is what its address would predict: an office and banking block empties at weekends, which is 5259 at 2.98 and, at a fifth of the volume, the university quarter at 4159; a retail and tourist frontage is busier on a Saturday afternoon than a Tuesday morning, which is 5161 at 0.79; a nightlife basin peaks after dinner on almost every day, which is 4556 at 0.90; and a mixed retail and through-traffic street, 5059 at 1.29, has no strong contrast either way. That last cell is where the calendar features later fail: the working-day indicator encodes exactly the contrast 5059 does not have.

Figure 2

Total Internet activity across the 100 by 100 grid on a log scale, north up with square 1 in the south-west corner, the five study cells marked, and a zoom on the central core. The three evaluation areas are neighbours inside one dark core, not separate districts, so their differing weekly regimes are the finding.

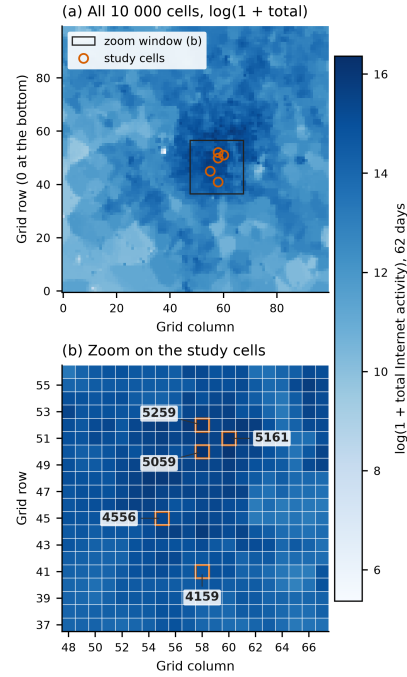
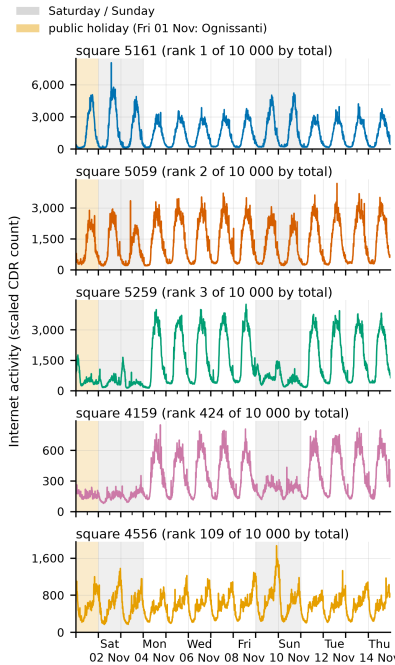


Figure 3

Internet activity over the first two weeks for the three highest-traffic cells and the two squares the brief names. Weekends are shaded grey and the 1 November public holiday amber. Square 5259 loses its daytime peak entirely on non-working days; square 5161 is busiest at weekends.



The heterogeneity costs the baselines immediately. A seasonal naive forecast at the daily lag scores MAE 470.3 on square 5259 against persistence at 76.0, because the daily lag crosses a regime boundary twice a week; the weekly lag, which preserves the day of week, more than halves that error to 210.2. One model per area, each with its own scaler, is the only defensible setup, and a shared architecture must cope with a weekend-heavy and a weekday-only cell at once.

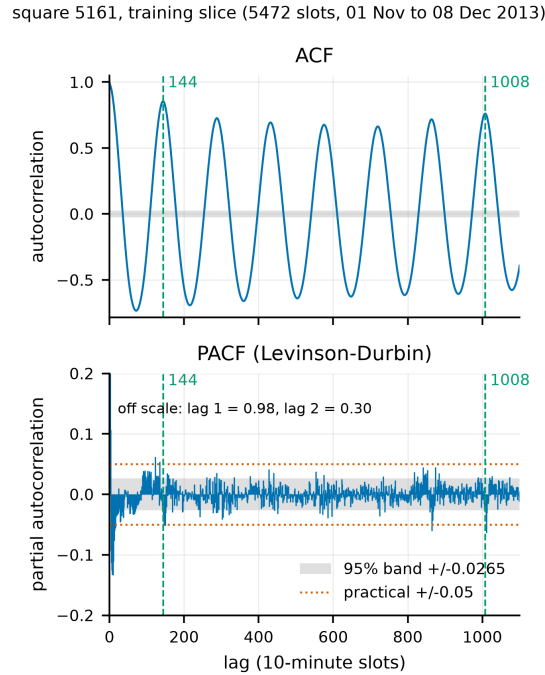
4.3 Temporal dependence and the sequence length

The autocorrelation function of the highest-traffic cell is dominated by the daily cycle, at 0.854 at lag 144 and still 0.761 at the weekly lag 1008 (Figure 4). The partial autocorrelation sets the input window: against a 95 percent band of ± 0.0265 at $n = 5472$, the last partial above a practical magnitude of 0.05 over lags 1 to 300 is lag 149, the partial at the daily lag itself is only -0.037 , and at lag 288 only -0.002 .

Beyond lag 300 exactly two partials still exceed that threshold anywhere in the first 1100, at lags 865 and 1011, the latter three slots past the weekly lag: a small and real weekly signal that the 144-slot window leaves to the model. Across lags 301 to 1100 only 5.9 percent exceed the band against the 5 percent expected by chance. Linear autoregressions corroborate the choice: on square 5161 validation MAE falls 115.1, 114.6, 109.2 across orders 6, 36 and 144, while order 216 improves the three-cell mean by 0.6 percent for 1.5 times the input. A window of 144 slots, one day, is a cost and accuracy trade, not a located optimum.

Figure 4

Autocorrelation and partial autocorrelation of square 5161 on the training slice. The daily lag 144 and the weekly lag 1008 are marked. Over lags 1 to 300 the partial autocorrelation is negligible beyond about lag 150, which sets the input window; the only exceedances further out are the two isolated partials at lags 865 and 1011 next to the weekly lag.



4.4 Seasonality, stationarity and the transform

An STL decomposition at period 144 [22] puts 89.5 percent of the variance in the daily seasonal component, 3.4 percent in the trend and 4.8 percent in the remainder. A multi-seasonal decomposition at periods 144 and 1008 splits that further, 88.0 percent daily and 11.1 percent weekly, while a classical centred moving-average decomposition puts the remainder at 11.8 percent (Table 2). The method changes that number by a factor of six, so the method is named wherever a share is quoted; what survives is the ordering, and a weekly component real but roughly eight times smaller than the daily one.

The 3.4 percent STL trend is mostly the weekly cycle, which a decomposition at period 144 cannot separate from a slow level. What either method calls irregular is not noise: the STL remainder is autocorrelated at 0.78 at lag 1, so most of the 4.8 percent is predictable one step ahead from the preceding value, which is the persistence floor of Section 6 seen from the decomposition side.

Both stationarity tests agree, though their nulls are opposites. The augmented Dickey-Fuller test rejects its unit-root null decisively on the raw series ($p = 3.3 \times 10^{-28}$); the KPSS test does not reject its level-stationarity null ($p > 0.10$), nor does its trend-stationary variant. The same verdict holds after a log transform, first differencing and seasonal differencing at lag 144. The series is therefore level-stationary over the training period and needs no differencing, but neither test addresses the daily cycle, which the input window handles instead.

Table 2

Variance shares in percent, $\text{var}(\text{component}) / \text{var}(\text{series})$, square 5161 on the training slice. Shares need not sum to 100 because the components are not orthogonal; the classical filter loses 72 rows at each end. Reading down the trend column shows what the period-144 methods misattribute: the weekly cycle has nowhere else to go. Reading down the remainder column under log1p is the transform decision of Section 5.4.

Series	Method	Trend	Daily	Weekly	Rem.	n
raw	STL (144)	3.4	89.5	-	4.8	5472
raw	MSTL (144, 1008)	0.4	88.0	11.1	2.0	5472
raw	classical (144)	4.0	84.4	-	11.8	5328
log1p	STL (144)	0.5	96.5	-	2.3	5472
log1p	MSTL (144, 1008)	0.1	95.9	3.2	1.2	5472
log1p	classical (144)	0.6	94.7	-	5.0	5328

Level-stationary is not the same as unchanging. The grid's Monday-to-Friday mean per slot falls 18.2 percent between the week of 4 November and the evaluation week, before any holiday, and the decline is uneven: square 5161 ends 0.1 percent below its starting level, 5059 and 5259 lose 7.4 and 7.7 percent, and 4159 loses 24.7 percent. Any model that learns a level will overshoot on the drifting cells, while persistence is immune to slow drift, part of why it is hard to beat one step ahead.

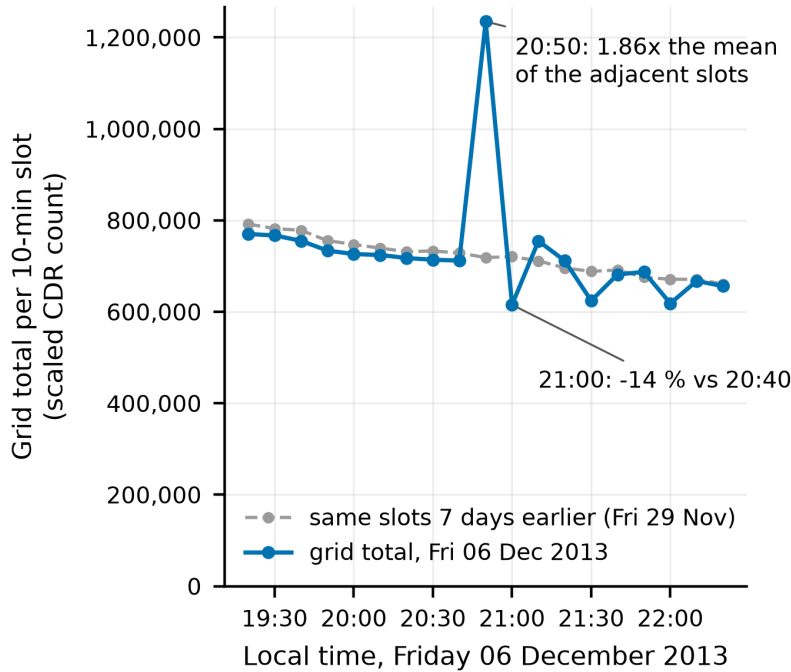
The transform decision rests on heteroscedasticity. The trailing 144-slot rolling mean and rolling standard deviation correlate at 0.973 on the raw training series and at 0.235 after a $\log(1+x)$ transform: the noise scales with the level. Both transforms are therefore carried into the experiments, and the remainder share falls under the log transform for every decomposition method tried.

4.5 One anomaly, and what it means for training

A scan of every slot against the median of the same slot of the week over the training rows flags exactly one event on the grid total: Friday 6 December at 20:50, where the grid jumps from 712 060 to 1 234 958 and falls back to 615 431 (Figure 5). That is 1.86 times the mean of its neighbouring slots and the largest single-step rise in the record, at 522 898 against a second-largest of 139 442. It is not local: 95.5 percent of the 10 000 cells are higher at 20:50 than at 20:40, and 76.9 percent exceed 1.5 times their own neighbouring slots. Only about a fifth of the excess is recovered afterwards: an additive burst, not traffic moved between slots.

Figure 5

Grid-wide Internet activity around Friday 6 December 20:50 against the same hour one week earlier. One slot reaches 1.86 times the level of its neighbours across 77 percent of cells, the largest single-step rise in the record.



Nothing in the preceding slots announces it, so no one-step-ahead model can predict it, and the rebound produces a second large error of opposite sign one step later. Row 5165 falls inside the training slice, so the reported metrics are unaffected, but the training loss is not: a squared-error objective sees one target at 1.7 times its context in nearly every cell. That is an argument for the gradient clipping Section 5 adopts.

The Christmas collapse is severe: on square 5161 the 24 to 26 December block falls 65.1 percent against the same weekdays a fortnight earlier, where the grid as a whole falls 36.5 percent, and 25 December alone falls 88.6 percent against the grid's 38.0. The brief's evaluation week of 16 to 22 December is therefore the last normal week of the record, and no result here is extrapolated past it. Minima over that week on the three study cells are 108, 182 and 161, so no percentage error divides by anything near zero.

5. Methodology

5.1 The forecasting task

At time t the algorithm maps the L most recent observations of one area to the next slot:

$$\hat{x}_a(t+1) = f(x_a(t-L+1), \dots, x_a(t)), \quad L = 144 \quad (1)$$

The 10-minute horizon is fixed by the brief; L is one day, chosen below.

5.2 Splits, and a validation week harder than the test week

Rows 0:5472 train (1 November to 8 December), 5472:6480 validate (Monday 9 to Sunday 15 December, one full week), 6480:7488 test (16 to 22 December, fixed by the brief). Chronological, because a forecaster is deployed forward in time.

The validation week is the busiest of the record on all three cells, 15.3 percent above the five training weeks by weekday mean on 5161, so every validation error below is inflated relative to its test counterpart: persistence scores MAE 116.8 against 92.8 on 5161, 99.9 against 81.5 on 5059 and 89.9 against 76.0 on 5259. The gap is a calendar effect, not overfitting.

5.3 Input representation and sequence length

The input is the 144 most recent slots of the target cell, one day, derived in Section 4.3 from the partial autocorrelation and corroborated there by an AR order sweep; Section 5.9 tests it a third time through the convolutional receptive field.

5.4 Normalisation

The scaler is fitted on the training rows only; models predict in the scaled space and the harness inverts once, centrally, before any metric. Standard and log1p-standard run as separate logged configurations, not one assumed. The choice rests on the training-slice heteroscedasticity of Section 4.4: the rolling mean and standard deviation decouple under the transform, and every decomposition method leaves a smaller remainder (Table 2). On validation the transform is close to a wash, lowering MAE on six of the nine fits by 1.1 to 4.0 percent and raising it on three, mean change -0.33 percent. It lowers evaluation-week MAE on all nine, a property of that week and not what the choice was made on.

5.5 Calendar features

Five features describe the target slot, all known at the forecast origin: time-of-day and day-of-week sine and cosine pairs, and a working-day indicator treating the public holidays observed in Milan inside the window (1 November, 7 and 8 December, 25 and 26 December, 1 January) as non-working. The regime finding of Section 4.2 rules out a plain weekend flag: on square 5259 the shape follows the working calendar, and Friday 1 November behaves as a Sunday. All three models get the same lags and features; one logged run ablates the features for all three, keeping the comparison at equal information.

5.6 The three models

The LSTM is one layer of 64 hidden units, its final hidden state concatenated with the five calendar features and mapped to one output, about 17 000 parameters. Gated recurrence carries a regime across many steps without a hand-specified lag structure [8], which the three regimes of Section 4.2 ask for and the closest prior work used [7]. It reads all 144 steps in sequence, forgoing the parallelism the other two have, and it has a known degenerate solution in which the gates saturate and the output tracks the last input, which would show up as persistence mimicry.

The temporal convolutional network is six residual blocks of two causal convolutions, kernel 3, dilations 1 to 32, 32 channels, with ReLU activations, weight normalisation and dropout 0.1, following [9]; about 35 000 parameters, the largest of the three trained models, so the comparison is matched neither on capacity nor on regularisation. Its receptive field of 253 steps answers the lag-149 cut-off of Section 4.3: everything the partial autocorrelation locates sits inside one parallel forward pass. The cost is a fixed field: nothing beyond 253 steps exists for this model at all.

The gradient-boosted trees, GBT in the tables and figures, are a HistGradientBoostingRegressor [23] with 300 boosting iterations, learning rate 0.05 and at most 31 leaves, taking those 149 columns as they come and dropping sequence structure. It is still a

sequential forecaster in the sense the research question uses; what it lacks is an ordering prior over those lags, exactly the inductive bias the comparison isolates [11]. Five low-cardinality calendar columns are the kind of split variable a tree handles natively, which should matter most on square 5259. Its output is piecewise constant, so it cannot go above the training maximum, and it stops on a different rule from the networks: scikit-learn holds out a random tenth of the training windows, which is optimistic against the chronological validation week the networks stop on because adjacent windows share 143 of their 144 values.

The predictions recorded in advance were: a few percent over persistence for the LSTM with persistence mimicry as its failure mode; LSTM-level accuracy for the TCN at 0.40 s per epoch against 0.58 s; and best or joint-best on 5259 for the trees, with no extrapolation above the training maximum.

5.7 Rejected alternatives

SARIMA was rejected on a measurement, not on principle: at seasonal period 144 on the 5472-point training slice one likelihood evaluation costs about 0.8 s undifferenced and close to four times that once the seasonal difference takes the state vector from 146 to 290, and a fit of the cheapest order got through about 150 evaluations in 120 s without converging. The linear family stays in through AR(144). A GRU is a variant of the same recurrent architecture, and Transformers lose to simple linear models at short horizons [13], the regime here.

5.8 Baselines

Seven reference forecasters were fitted first, in EXP-000: persistence, seasonal naive at the daily lag of 144 and the weekly lag of 1008, a four-day time-of-day mean and linear autoregression at orders 6, 36 and 144. Three carry through into the reported run and into every table of Section 6, persistence, the daily seasonal naive and AR(144), each refitted on the same 144-slot window as the trained models. Persistence reaches R^2 0.9902 and WAPE 6.42 percent on 5161, and AR(144) lowers its MAE by 8.4 percent and its RMSE by 2.9 percent, the headroom the trained models compete for.

Copying a past value commutes with a monotone transform, so persistence and the seasonal naive are scale invariant, but the autoregression is fitted in whatever space the run uses: in Section 6 a linear map of the log1p-standard series. It also receives the 144 lags only, 145 parameters, without the calendar block the three trained models get, so its margins bound the architectural gain from above: part of the leads reported in Section 6.1 could be the features, which Section 6.4 measures at 0.8 to 2.6 percent of MAE.

5.9 Training and tuning

Adam at 1e-3, batch 128, gradient clipping at 1.0, up to 100 epochs, early stopping on validation MSE with patience 10 and best-epoch weights restored, and the learning rate halved once six consecutive epochs pass without validation improvement. Loss and early stopping work on the scaled target, every reported metric in original units. The optimiser, batch size, clipping threshold and loss were held at these values in all 69 neural fits, so every logged change is attributable to representation or capacity, never to a moving optimiser; the price is that none of the four is justified by an experiment of its own. Seed 42 while iterating, three seeds for the final configuration, reported as mean and standard deviation.

Every model is retrained per area with its own scaler, because the cells run on different regimes; tuning has two scopes. The input representation was decided across all three cells by manual one-variable changes (Table 3, EXP-001 to EXP-003), the rationale written before each:

with under 10 percent of headroom over the linear baseline, why a change helps is the finding, not the change itself. Capacity was then swept on the highest-traffic cell alone (EXP-010 to EXP-018), as a one-variable grid at three seeds, nine runs, the TCN over five, six and seven levels and 16, 32 and 64 channels, the LSTM over 32, 64 and 128 hidden units and the tree ensemble over 15, 31 and 63 leaves, selected on validation.

Table 3

The run-by-run trail behind this section, one row per run or run group.

experiments/EXPERIMENT_LOG.md holds each row's reasoning, written before the run, and each run directory holds its own config, metrics and log. The validation column reads the TCN on square 5161 throughout, except EXP-000 (linear AR(144) on the same cell) and EXP-006 and EXP-007 (the TCN on square 5059); it identifies a row rather than ranking rows against each other.

Run	What changed	Val MAE	What it settled
EXP-000	seven reference forecasters, 1008-slot window, unscaled	109.16	the floor, fixed before any architecture was chosen
EXP-001	first trained run: window 1008 to 144, standard scaling, calendar on	102.19	all three clear persistence on 5161 and 5259; on test MAE only the TCN clears AR(144) there
EXP-002	standard to log1p-standard	98.10	adopted, on the training-slice evidence of Section 4.4
EXP-003	calendar features off	99.52	kept on; the two exceptions are both on square 5059
EXP-004, 005	three seeds on 5059, block on and off	-	invalid: sd exactly 0.000 exposed a seed that never reached the model
EXP-006, 007	the same pair, after the fix	88.30, 83.28	the 5059 effect survives three seeds and becomes the failure case
EXP-008	the reported configuration: three areas, three seeds	100.11	every number in Section 6
EXP-010 to 018	capacity grid, one variable per run, three seeds	100.11	confirms the convolutional depth; two models left at a default

The convolutional model's reported configuration is the validation pick, and it confirms the window the exploratory analysis derived: five levels reach only 125 of the 144 slots and cost 1.5 percent of validation MAE, while seven reach 509 and buy nothing for 13 percent more time per epoch. The recurrent and tree models were left at configurations validation does not select. A wider tree ensemble, 63 leaves, improves validation MAE by 2.3 percent, beyond any seed spread in that block; 128 hidden units improve it by 0.6 percent, which is inside one. Neither was adopted, because the reported campaign and its tests were already closed and the larger gain would change no conclusion. The shortfall is stated in Section 7.

5.10 Metrics

The brief requires MAE, RMSE and MAPE; WAPE, MASE and R^2 are reported beside them. MASE takes its denominator from the training-period seasonal-naïve error [14]; scaling by the scored window inflates every MASE, by a factor of 1.38 on square 5161, which made a pure seasonal-naïve forecast read 1.34 there instead of 0.975. MAPE is well defined here, traffic never

falling below 108 in the evaluation week, so the epsilon guard never binds. It still over-weights the 01:00 to 06:00 slots by up to 1.3 times their share of the week, hence WAPE beside it.

5.11 Separating two forecasters

A margin of two or three units between table columns is not a result until it is tested, so every pairwise claim in Section 6 carries a Diebold-Mariano test on the paired absolute-error differentials of the 1008 evaluation slots [24]. The differential is taken between the means over the three seeds of each slot's absolute error, so the quantity tested is exactly the seed-mean MAE the tables print; testing the mean of the three forecasts instead would score a three-member ensemble that appears in no table here, and would widen the convolutional model's margin on square 5161 from 2.85 to 5.53 units.

Serial correlation in the loss differentials rules out the uncorrected variance (Ljung-Box $p < 0.002$ for all fifteen comparisons), so the variance of the mean uses a Bartlett kernel at lag 15 with the Harvey-Leybourne-Newbold small-sample correction [25] against a standard normal reference. Only two of fifteen conclusions change anywhere between lag 0 and lag 144, and both are named where they arise. Three comparison families were fixed before the tests ran, with Holm-adjusted values within each. The reordering claim is a conjunction, not a selection, so it takes an intersection-union test and no multiplicity correction [26].

5.12 Evaluation convention and timing

Scoring implements Equation (1) as rolling-origin one-step forecasting with observed history [15]: 1008 origins, none recursive. Inputs may reach back before 16 December because they are available at the forecast origin; targets never do, so no held-out target is an input to its own forecast. Validation follows the same convention.

Training time is fit wall and CPU seconds with epochs and seconds per epoch, since under early stopping a bare wall time is not representative. Execution time is the median of 20 warm repetitions over the 1008 evaluation windows, plus a single-window latency, the device synchronised before the timer stops: a first call at a new batch shape on MPS is about 30 times slower than warm.

6. Results and Discussion

All numbers below come from run EXP-008: three architectures on three areas at three seeds each, log1p-standard scaling, calendar features on, evaluated on 16 to 22 December against reference forecasters fed the identical 144-slot window. Trained models are reported as the seed mean, and every pairwise margin is tested on the paired error differentials of that seed-averaged loss path.

6.1 Per-area accuracy

Table 4

Square 5161, the highest-traffic cell. Bold marks the lowest point estimate in each column, not a demonstrated win; the standard deviation over the three seeds follows the MAE in brackets. The TCN's 2.85-unit lead over the linear model is inside its own 6.65 seed spread and inside the test's interval, $[-9.01, +3.31]$ at $p = 0.36$. Nothing here separates the two.

Model	MAE	MAPE (%)	RMSE	WAPE (%)	MASE	R ²
LSTM	84.42 (1.41)	8.02	129.35	5.84	0.243	0.9910
TCN	82.13 (6.65)	7.83	124.35	5.68	0.236	0.9916

Model	MAE	MAPE (%)	RMSE	WAPE (%)	MASE	R ²
GBT	87.08 (1.87)	8.41	130.39	6.03	0.251	0.9908
Persistence	92.80	9.19	134.88	6.42	0.267	0.9902
Seasonal naive (daily)	338.59	25.94	619.04	23.43	0.975	0.7934
Linear AR(144)	84.98	7.74	130.92	5.88	0.245	0.9908

Table 5

Square 5059. The linear autoregression beats all three trained models on every metric, by margins of 7.1 to 16.0 percent of the trained model's MAE that exceed every seed spread on the cell. They are also the only trained-versus-linear margins in the study that survive a test, and only against the TCN; the values are in Figure 6 and VI.A.

Model	MAE	MAPE (%)	RMSE	WAPE (%)	MASE	R ²
LSTM	71.84 (2.97)	6.77	103.68	5.68	0.266	0.9878
TCN	78.88 (3.58)	7.22	115.32	6.23	0.292	0.9849
GBT	71.32 (0.53)	6.79	102.68	5.64	0.264	0.9880
Persistence	81.52	7.96	114.38	6.44	0.302	0.9851
Seasonal naive (daily)	171.74	18.02	245.87	13.57	0.635	0.9313
Linear AR(144)	66.25	6.30	96.67	5.24	0.245	0.9894

Table 6

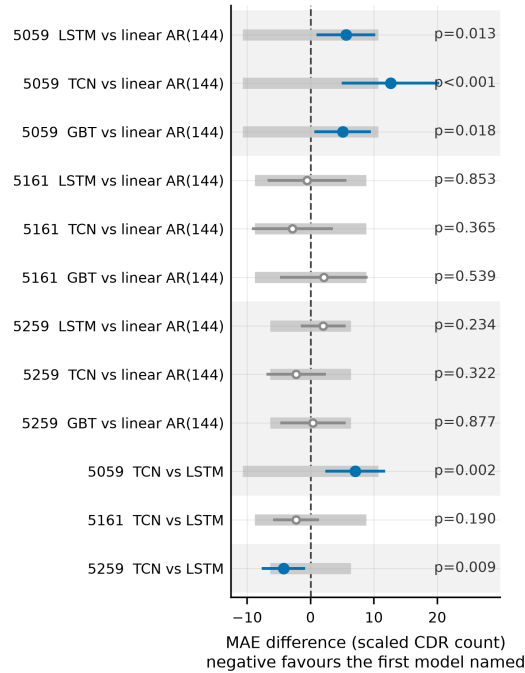
Square 5259, the weekday-only cell. Bold marks the lowest point estimate; seed standard deviation in brackets after the MAE. The TCN's 2.25-unit lead over the linear model clears its seed spread of 0.46 but not the week's sampling variation: the interval is $[-6.72, +2.21]$ at $p = 0.32$. Seed stability and separability are different questions, and this cell answers the first yes and the second no. What does hold here is the TCN's 4.26-unit margin over the LSTM, $p = 0.009$, the reverse of the ordering on square 5059.

Model	MAE	MAPE (%)	RMSE	WAPE (%)	MASE	R ²
LSTM	65.70 (0.29)	7.11	94.13	5.19	0.125	0.9930
TCN	61.43 (0.46)	6.65	89.29	4.86	0.117	0.9937
GBT	64.08 (0.51)	6.92	92.93	5.06	0.122	0.9931
Persistence	75.97	8.11	109.58	6.00	0.145	0.9904
Seasonal naive (daily)	470.32	71.62	861.62	37.18	0.898	0.4093
Linear AR(144)	63.69	6.66	93.74	5.03	0.122	0.9930

Figure 6

Difference in mean absolute error between the paired forecasters of the three results tables, with 95 percent Newey-West intervals at lag 15 over the 1008 evaluation slots. Negative favours the first model named; filled markers are the comparisons with $p < 0.05$. The grey bar behind each area is the minimum detectable difference at 80 percent power of that area's TCN-versus-linear

comparison, 10.67, 8.81 and 6.38 units for squares 5059, 5161 and 5259. The two comparisons the study's headline once rested on are among those whose intervals cross zero.



The ranking changes with the area. The TCN is best on squares 5161 and 5259 and worst of the three on 5059, the only trained model there that fails to beat persistence on RMSE. The LSTM and the tree model swap places between cells. Answering the research question from one area, which the brief allows, would have been confident and wrong. The TCN beats the LSTM by 4.26 units on 5259, $p = 0.009$, and loses to it by 7.03 units on 5059, $p = 0.002$ (Figure 6); as a conjunction the intersection-union p is 0.004, and it rejects at every lag from 0 to 144. The margins over the linear model do not survive: 2.85 units on 5161 at $p = 0.36$ and 2.25 on 5259 at $p = 0.32$. One week on these cells resolves 8.81 and 6.38 units, 10.4 and 10.0 percent of the linear model's MAE, so margins of 3.4 and 3.5 percent were never within reach; detecting them would take about 67 and 56 days. The ordering is real and the podium is not.

A 145-parameter linear model is the real competitor, and on average it wins. Linear AR(144) wins on square 5059 on all six metrics, beating the best trained model there by 7.1 percent of that model's MAE, and it is the only cell where a trained-versus-linear margin separates from zero ($p = 0.0009$ against the TCN, 0.008 after Holm adjustment). On 5161 and 5259 it loses to the TCN by 3.4 and 3.5 percent. Aggregate the three cells and the advantage disappears: the per-area best trained model averages 71.63 MAE against the linear model's 71.64, a difference of 0.015 percent. Architecture by architecture the linear model is ahead of all three, by 3.3 percent over the LSTM, 3.5 percent over the TCN and 3.5 percent over the trees. Per-cell selection buys that parity without the test set: on the validation week the same model ranks first on each cell. This matches the critical literature [12], [13], [11].

The best-performing architecture, and the grounds for naming it. Of the three the study names the dilated convolutional network, on one quantitative ground and one qualitative one. Quantitatively it holds the lowest error on two of the three cells, and it is in both halves of the one architecture-versus-architecture comparison the study pre-specified. Qualitatively that win is exactly where Section 4 says it should be. Square 5259 is the cell whose daily shape

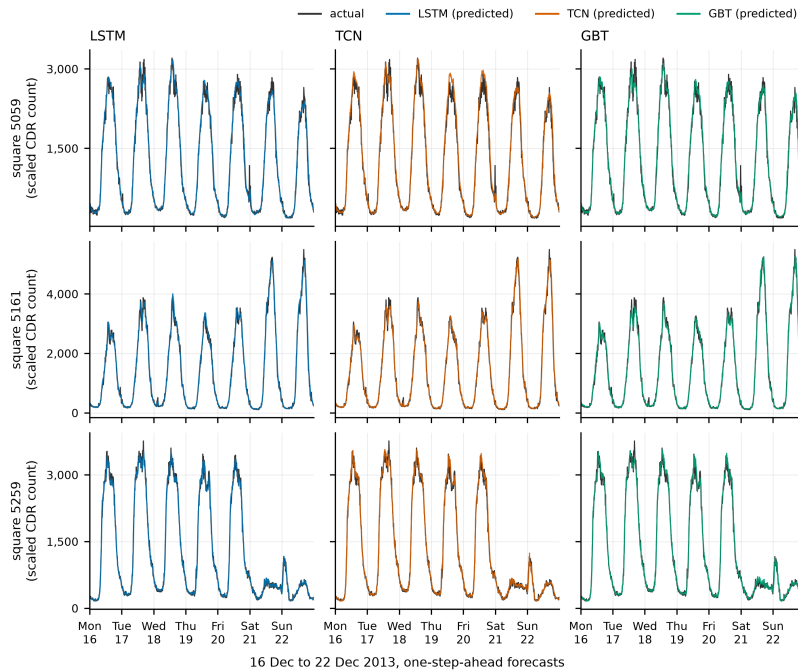
changes between regimes rather than merely shrinking: its mean working-day and non-working-day profiles correlate at -0.28 , against 0.95 on 5161 and 0.91 on 5059, and 59 percent of its profile variance lies outside its own mean profile, against 6 and 9 percent. On that cell, knowing which of two shapes the previous day had is worth something, and a receptive field of 253 steps reads it off the window in a single pass, where a recurrent state must carry it through 144 sequential updates and a linear map has one fixed set of coefficients for both regimes. The same reading accounts for the loss on 5059: a shape that barely changes leaves no regime to identify, and the calendar block that encodes the regime becomes the noise of Section 6.4.

Two qualifications belong with the name. The convolutional model's second win, 2.85 units on square 5161 at $p = 0.36$, is a point estimate rather than a result, and the argument above does not extend to it. And the best of three architectures is not a recommended forecaster: averaged over the three cells none of them betters the 145-parameter linear autoregression.

The daily seasonal naive is not a weak baseline, it is a wrong one. It scores 338.6, 171.7 and 470.3 against persistence at 92.8, 81.5 and 76.0. The failure is worst on square 5259, the cell whose regime changes twice a week: at a one-step horizon a ten-minute-old observation beats a day-old one, and the age of the information dominates the seasonal structure.

Figure 7

One-step-ahead forecasts against observed traffic for each model and area over the evaluation week, drawn for seed 42 rather than the seed mean, which would be a three-member ensemble appearing in no table here. Rows are areas, columns are models, and the vertical scale is shared within each row. All nine panels track the daily cycle closely; the visible errors are at the sharp afternoon ramps and, on square 5259, in the flat weekend of 21 and 22 December.



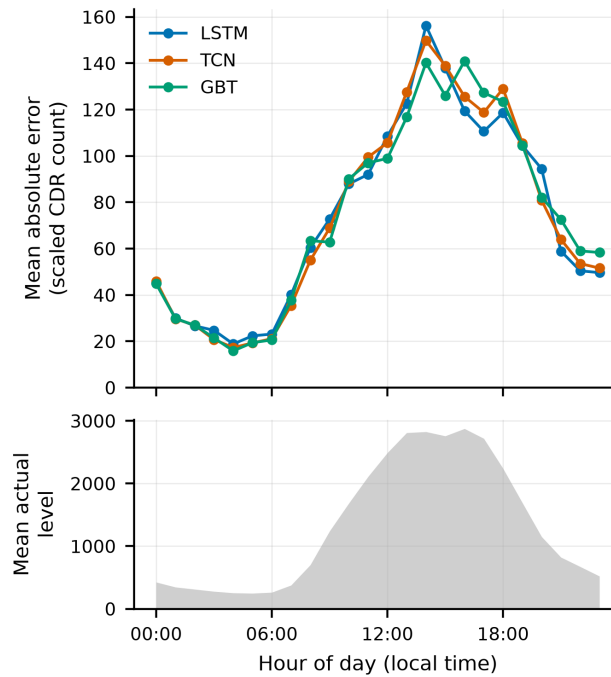
6.2 Why the errors sit where they do

Figure 7 shows nine forecasts that are, at this resolution, hard to distinguish from the truth, which is why a table of R^2 values above 0.98 carries so little information here. The differences live in the ramps.

Error is strongly heteroscedastic across the day (Figure 8). For persistence on square 5161 the mean absolute error at 13:00 is 164 against 19 between 02:00 and 05:00, a factor of 8.7, and the busiest 5 percent of slots in the evaluation week carry about a tenth of all absolute error on each of the three cells, twice their share of the week. Every model inherits this shape. A headline MAE is therefore dominated by a few hours and describes peak-hour behaviour more than average behaviour. MAPE, which weights the quiet 01:00 to 06:00 slots at up to 1.3 times their share of the week, is the metric least aligned with what an operator provisioning capacity cares about, which is why WAPE is reported beside it.

Figure 8

Mean absolute error by hour of day, pooled over the three areas and averaged over the three seeds of each trained model, with the mean observed level beneath it. Error tracks the level: the models are accurate at night because the series is flat at night.



6.3 Computational cost

Table 7

Training and execution time, averaged over the three areas and three seeds. Training is the wall time of one fit under early stopping, reported with the epochs actually run; the area-to-area spread behind those means is wide, 25.0 to 29.9 s for the LSTM and 38.4 to 53.1 s for the TCN. Batch is the median of 20 warm repetitions over all 1008 evaluation slots; latency is the median

of 50 single-window predictions, the device synchronised before each timer stops. Hardware: Apple M1 Pro, 8 cores, 16 GB, macOS, PyTorch 2.14 with MPS.

Model	Params	Device	Epochs	Train (s)	Batch (ms)	Latency (ms)
LSTM	17 222	mps	42	28.2	14.10	3.332
TCN	34 758	mps	34	44.1	20.89	3.305
GBT	3 795	cpu	122	2.1	9.21	7.118
Persistence	0	cpu	-	0.00	0.02	0.002
Linear AR(144)	145	cpu	-	0.02	0.06	0.004

The cost spread in Table 7 is three orders of magnitude wider than the accuracy spread. Training the TCN takes 44 s against 0.02 s for the linear autoregression, and batch inference over a week of slots 20.9 ms against 0.06 ms, for 3.4 percent of MAE at best and a loss at worst. The tree model is the compromise: 2.1 s to fit, 9.2 ms to predict, within about 3 percent of the LSTM on every cell and ahead of it on two.

Single-window latency inverts the batch ordering: among the trained models the tree model is the fastest in a batch and the slowest per window, 7.1 ms against 3.3 ms, because its per-call overhead does not amortise. The first MPS call at a new batch shape runs about thirty times slower than a warm one, so a single unsynchronised timing would have reported the neural models as roughly an order of magnitude slower than they are. Both are artefacts of how the question is asked, not properties of the models.

6.4 Failure case: the calendar features, square 5059, and two days in December

The clearest failure is not a model losing to a baseline but an input representation that helps two cells and hurts a third. Switching off the five calendar features costs 0.8 to 2.6 percent of MAE on seven of the nine model-area pairs on a single seed, inside the seed spread of two of the three architectures and so weak on its own. Both exceptions are on square 5059, and the larger is not weak at all. Repeated over three seeds, the TCN on 5059 improves from 78.88 to 68.42 MAE without the features, a gain of 13 percent, and its standard deviation across seeds falls from 3.58 to 0.97. The features are worse on validation as well as on test, so the model trains worse; early stopping is not the culprit.

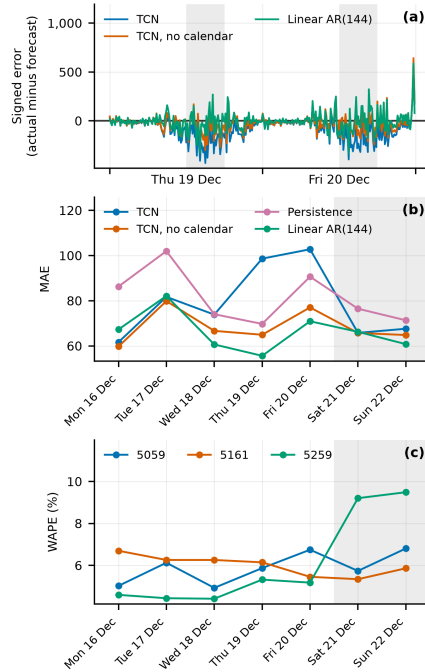
That deficit is not spread across the week (Figure 9). Outside Thursday 19 and Friday 20 December the TCN and the linear model are level, 70.14 against 67.43 ($p = 0.29$), and over those five days the network beats persistence by 11.90 units ($p < 0.0001$). On the two days it scores 100.72 against persistence's 80.21 and loses significantly ($p = 0.010$): a trained convolutional network beaten by repeating the last observation. Between 79.5 and 95.5 percent of its net excess over the linear model falls on those two days in each of the three seeds. This is the period the brief asks for, and the same event as the ablation above.

The cause is not turbulence: Thursday is the calmest weekday of the week, and persistence, whose absolute error is by definition the mean absolute ten-minute change, scores 69.8 there against 87.4 on Monday to Wednesday. It is not lag either, since the residual's correlation with the first difference of the series is lower for the TCN than for the linear model, 0.617 against 0.710. It is bias in the afternoon plateau. Over the two days the TCN's signed error averages -78.4 against the linear model's -5.2 , and squared bias accounts for 30.4 percent of its mean squared error against 0.3 percent for the linear model. Withholding the calendar block

removes 34.2 percent of the Thursday error and 25.0 percent of the Friday error, and 81.2 percent of the whole calendar penalty on this cell falls on those two days.

Figure 9

Per-period error on the evaluation week. (a) Signed error, actual minus forecast, on square 5059 over Thursday 19 and Friday 20 December; the shaded bands mark 12:00 to 18:00, where the TCN sits below zero and so over-predicts. (b) Per-day MAE on square 5059, mean over three seeds, with the calendar-off control. (c) Per-day WAPE by area, mean over the five non-seasonal forecasters. Saturday and Sunday are shaded in both lower panels.



Square 5059 has the weakest calendar signal of the three, though not in the way the weekday-to-weekend ratios suggest: 1.29 against 0.79 and 2.98 puts it between the other two. The quantity that matters is what the working-day flag adds on top of time of day, because time of day is already in the window. Regressed on the 144 time-of-day slots alone, the training series reaches R^2 0.884 on 5059, 0.839 on 5161 and 0.438 on 5259; adding the working-day indicator and its interaction lifts those to 0.945, 0.942 and 0.925. The flag is worth 0.06 of R^2 on 5059 against 0.10 on 5161 and 0.49 on 5259, so on this cell it carries three fifths of what it carries next door and an eighth of what it carries on 5259. The features there are close to noise, and the two models that degrade are the two fed that noise on the cell where it carries least.

The architectural half is subtler, and the obvious explanation is wrong. Both neural models concatenate the calendar block to a single summary vector and pass it through one linear head, so neither folds it into the recurrence. The asymmetry is in what the summary can do about it. The LSTM's 64-unit hidden state is free to shrink its own scale relative to a fixed calendar term, and the tree model can award those five columns no splits, which it largely does. The TCN's last-step channel vector is the output of a fixed convolutional stack whose scale is tied to the input, leaving the head less freedom to attenuate a term it should ignore. The seed instability, 3.58 against 0.97, is consistent with that reading, which remains a hypothesis about optimisation rather than a demonstrated mechanism.

The features were kept on for every model. Turning them off for one architecture on one cell would buy about 13 percent of MAE there and destroy the equal-information comparison the study rests on. An input representation justified by an exploratory finding on one cell, here the night-time peaks of square 5259 on non-working days, does not automatically transfer to a neighbouring cell 500 m away.

Panel (c) of Figure 9 carries a warning about the metric. On square 5259 every forecaster roughly doubles its WAPE at the weekend, from 4.8 percent on Monday to Friday to 9.4 percent on Saturday and Sunday, which invites the reading that the weekend is hard. Absolute error moves the other way over the same slots, from 76.2 to 41.2, because the level collapses from 1595 to 441: the denominator falls faster than the error does. No model is significantly worse than any other there. The weekend on 5259 is a property of the cell and of the choice of metric, not a failure of any forecaster.

A second failure is shared by every model and fixable by none. The grid-wide spike on 6 December at 20:50, described in Section 4, is unannounced by anything in the preceding window, so a one-step-ahead forecaster must miss it and then miss again in the opposite direction when the series falls back. It lies in the training period, outside the evaluation week, but it is the clearest illustration of the ceiling on this task: a model that has learned the daily cycle perfectly still cannot forecast an event that the recent past does not foreshadow.

6.5 Personal considerations and margins for improvement

Across the three cells the gap between a well-tuned linear autoregression and three modern sequence models lies between minus 7.7 and plus 3.5 percent of the linear model's mean absolute error. Persistence reaches R^2 0.99 because ten-minute traffic is smooth, and almost all of the remaining error is concentrated in a few ramp hours per day. A model can only compete for that residue, the component Xu et al. separate out and declare unpredictable [1]; the measurement here is that the predictable part is largely already in the previous observation.

Section 5 recorded an expectation for each model before any of them was run, and two of the three were wrong. The LSTM was expected to clear persistence by a few percent and it did, by 9 to 14 percent, though persistence mimicry was not the failure mode that appeared. The TCN was expected to match the LSTM at 0.40 s per epoch against 0.58 s; it matched on accuracy but ran at 1.28 s per epoch against 0.67 s, 1.9 times slower, not faster. The tree model was expected to be best or joint-best on square 5259 and came second of the three trained models there, while winning on 5059 instead, the cell nobody predicted.

7. Conclusion and Future Work

The research question asked how sequential models compare for one-step-ahead mobile traffic forecasting, and how that comparison varies across areas. They compare closely, and the ordering is not stable. A dilated temporal convolutional network had the lowest error on two of the three cells, by 3.4 and 3.5 percent of MAE over a linear autoregression on 144 lags, but neither lead separates from zero ($p = 0.36$ and 0.32), and one week on these cells resolves only differences of about 10 percent. The reordering does separate: the convolutional network beats the recurrent one on square 5259 and loses to it on 5059, jointly at $p = 0.004$. The exploratory analysis predicts that pair: 5259 is the one cell whose daily shape changes between regimes instead of shrinking, which is what a 253-step receptive field is for, while the shape of 5059 barely moves, making its calendar features noise. On 5059 the linear model beat all three trained architectures on every metric, by 7.1 to 16.0 percent; the widest is the study's one trained-versus-

linear gap that a test separates from zero ($p = 0.0009$). Aggregated, the advantage vanishes: the per-area best trained model averages 71.63 MAE against 71.64 for the linear one, and parity is bought only by picking the winning architecture per cell, at between 96 and 2400 times the training time.

Information age dominates seasonal structure at this horizon: a forecast from a ten-minute-old observation reaches R^2 0.99, one from a day-old observation at the same time of day between 0.41 and 0.93, so the daily cycle, 89.5 percent of the variance under STL at period 144, is already contained in the recent past. Heterogeneity between cells 500 m apart is enough to reorder the models, so cross-area evaluation is not a formality. An input representation can also harm a cell whose signal it was not derived from.

The evaluation covers one week on the three most extreme cells of a heavy-tailed distribution, so nothing here generalises to a typical cell without retesting. A defect that let both trainable models re-seed themselves meant several early runs repeated a single seed; the reported configuration was rerun at three seeds after the fix, but the intermediate comparisons carry unmeasured variance. Capacity was swept only after the campaign closed, and it found two of the three architectures at configurations validation does not select, so they carry a reasonable configuration rather than the best one tried; the convolutional model, whose configuration the sweep does select, may still be under-tuned on the cell where it failed, since the sweep ran on a different cell. Finally, the memory and timing evidence come from two different machines.

Each limitation suggests a next step. Feeding the calendar block at every time step, instead of concatenating it to a single summary vector as both neural models do, would test whether the convolutional failure is about the head's inability to attenuate a term it should ignore. Evaluating at horizons of one hour and one day would show whether the small gaps found here are a property of the models or of the horizon. Using neighbouring cells as covariates [5] would ask whether the regime differences documented in Section 4 are predictable from spatial context. Repeating the comparison on cells from across the traffic distribution, not its extreme tail, would establish whether the ranking instability is general or a feature of the grid's busiest corner.

8. Use of AI Assistance

The assignment permits AI tools and requires significant use to be disclosed.

The tool was Claude Code, Anthropic's command-line coding assistant, running Claude Opus. It wrote the first draft of most of the Python package, including the streaming ingestion, the windowing and scaling helpers, the training loop, the three model modules and the test suite, and it drafted prose for this report from the author's working notes. It was also used as a reviewer: independent audit passes were run over the repository, and the numerical claims of Sections 4 and VI were recomputed from the stored matrix by a further pass instructed to refute the first, not to confirm it.

On the prose: the argument of each section, the decisions it defends and every interpretive claim were written by the author first, in the working notes and the decision register the assistant then drafted from. Every drafted paragraph was rewritten by the author against the stored numbers, and every quantitative claim was checked against the artefact that produces it. The abstract and this section were written by the author directly.

Review mattered more than generation. Three of the four defects were in the harness: metrics scored in the scaled space against ground truth in the original scale, which would have reported MAE 1445 in place of 92.8 for persistence; non-finite predictions dropped silently from

the average; and an unrecognised scaling name falling through to an identity transform instead of raising. All three passed a suite of 43 component tests that never imported the module assembling them. The fourth was found by an experiment: both trainable models re-seeded themselves inside their fitting routine, and the symptom was a standard deviation of exactly zero across three nominally different seeds. Each is recorded in `experiments/EXPERIMENT_LOG.md` and pinned by an integration test that fails if the fix is reverted.

Every figure and table cell is produced by a script in the repository and can be regenerated from a clean clone; the numbers quoted in prose are copied from those tables, except the descriptive statistics of Section 4, which `scripts/make_eda_stats.py` writes to `reports/tables/eda_stats.md`. The interpretations, the argument and the conclusions are the author's own.

8.1 Resources

The source code, the experiment log with every run's configuration and metrics, and the scripts that regenerate every figure and table are in the project repository [27], and the video presentation accompanying this report is at [28].

References

- [1] F. Xu et al., "Big data driven mobile traffic understanding and forecasting: A time series approach," *IEEE Trans. Services Comput.*, vol. 9, no. 5, pp. 796-805, 2016, doi: 10.1109/TSC.2016.2599878.
- [2] C. Zhang and P. Patras, "Long-term mobile traffic forecasting using deep spatio-temporal neural networks," in *Proc. 18th ACM Int. Symp. Mobile Ad Hoc Netw. Comput. (MobiHoc)*, Los Angeles, CA, USA, 2018, pp. 231-240, doi: 10.1145/3209582.3209606.
- [3] G. Barlacchi et al., "A multi-source dataset of urban life in the city of Milan and the Province of Trentino," *Sci. Data*, vol. 2, Art. no. 150055, 2015, doi: 10.1038/sdata.2015.55.
- [4] L. Chen, D. Yang, D. Zhang, C. Wang, J. Li, and T.-M.-T. Nguyen, "Deep mobile traffic forecast and complementary base station clustering for C-RAN optimization," *J. Netw. Comput. Appl.*, vol. 121, pp. 59-69, 2018, doi: 10.1016/j.jnca.2018.07.015.
- [5] J. Wang et al., "Spatiotemporal modeling and prediction in cellular networks: A big data enabled deep learning approach," in *Proc. IEEE INFOCOM*, Atlanta, GA, USA, 2017, pp. 1-9, doi: 10.1109/INFOCOM.2017.8057090.
- [6] C. Zhang, H. Zhang, D. Yuan, and M. Zhang, "Citywide cellular traffic prediction based on densely connected convolutional neural networks," *IEEE Commun. Lett.*, vol. 22, no. 8, pp. 1656-1659, 2018, doi: 10.1109/LCOMM.2018.2841832.
- [7] H. D. Trinh, L. Giupponi, and P. Dini, "Mobile traffic prediction from raw data using LSTM networks," in *Proc. IEEE 29th Annu. Int. Symp. Pers., Indoor Mobile Radio Commun. (PIMRC)*, Bologna, Italy, 2018, pp. 1827-1832, doi: 10.1109/PIMRC.2018.8581000.
- [8] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735-1780, 1997, doi: 10.1162/neco.1997.9.8.1735.
- [9] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," 2018, arXiv:1803.01271.
- [10] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, no. 5, pp. 1189-1232, 2001, doi: 10.1214/aos/1013203451.
- [11] S. Elsayed, D. Thyssens, A. Rashed, H. S. Jomaa, and L. Schmidt-Thieme, "Do we really need deep learning models for time series forecasting?," 2021, arXiv:2101.02118.
- [12] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "Statistical and machine learning forecasting methods: Concerns and ways forward," *PLOS ONE*, vol. 13, no. 3, Art. no. e0194889, 2018, doi: 10.1371/journal.pone.0194889.
- [13] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are transformers effective for time series forecasting?," *Proc. AAAI Conf. Artif. Intell.*, vol. 37, no. 9, pp. 11121-11128, 2023, doi: 10.1609/aaai.v37i9.26317.
- [14] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," *Int. J. Forecasting*, vol. 22, no. 4, pp. 679-688, 2006, doi: 10.1016/j.ijforecast.2006.03.001.

- [15] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: <https://otexts.com/fpp3/>
- [16] Telecom Italia, "Telecommunications - SMS, Call, Internet - MI," Harvard Dataverse, 2015, doi: 10.7910/DVN/EGZHFV.
- [17] Telecom Italia, "Milano Grid," Harvard Dataverse, 2015, doi: 10.7910/DVN/QJWLFU.
- [18] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357-362, 2020, doi: 10.1038/s41586-020-2649-2.
- [19] The pandas development team, "Scaling to large datasets, pandas 3.0.5 user guide," 2026. [Online]. Available: https://pandas.pydata.org/docs/user_guide/scale.html
- [20] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf.*, 2010, pp. 56-61, doi: 10.25080/Majora-92bf1922-00a.
- [21] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 2019, arXiv:1912.01703.
- [22] R. B. Cleveland, W. S. Cleveland, J. E. McRae, and I. Terpenning, "STL: A seasonal-trend decomposition procedure based on loess," *J. Off. Stat.*, vol. 6, no. 1, pp. 3-73, 1990.
- [23] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825-2830, 2011. [Online]. Available: <https://www.jmlr.org/papers/v12/pedregosa11a.html>
- [24] F. X. Diebold and R. S. Mariano, "Comparing predictive accuracy," *J. Bus. Econ. Statist.*, vol. 13, no. 3, pp. 253-263, 1995, doi: 10.1080/07350015.1995.10524599.
- [25] D. Harvey, S. Leybourne, and P. Newbold, "Testing the equality of prediction mean squared errors," *Int. J. Forecasting*, vol. 13, no. 2, pp. 281-291, 1997, doi: 10.1016/S0169-2070(96)00719-4.
- [26] R. L. Berger, "Multiparameter hypothesis testing and acceptance sampling," *Technometrics*, vol. 24, no. 4, pp. 295-300, 1982, doi: 10.1080/00401706.1982.10487790.
- [27] S. Serein, "milan-traffic-forecasting: source code for this study," GitHub repository, 2026. [Online]. Available: <https://github.com/supserrr/milan-traffic-forecasting>
- [28] S. Serein, "Comparative analysis of sequential models for mobile network traffic forecasting: video presentation," 2026. [Online]. Available: [INSERT VIDEO URL BEFORE SUBMISSION]